

Day 0 - Before You Write Any Code: The Absolute Basics

Day 0 — Before You Write Any Code: The Absolute Basics

This day exists because everything after it assumes you know what a "terminal" is, what "running a script" means, and how to make sense of an error message. If you've never programmed before, read this first — it will make Day 1 dramatically less overwhelming. If any of this is already familiar, skim it and move on.

What is a computer program, really?

A computer only ever does one thing: it follows instructions, one at a time, incredibly fast. A "program" (also called "code" or a "script") is just a list of instructions written in a language the computer can understand.

Python is one such language — a way of writing instructions that a specific program (the "Python interpreter") reads and carries out, step by step, from top to bottom.

When people say "I wrote a Python script," they mean: they typed a list of instructions into a text file, and that file can be handed to the Python interpreter, which will read it and do exactly what it says — no more, no less, and never anything you didn't literally tell it to do. This is the single most important mental habit to build as a beginner: **the computer does exactly what your code says, not what you meant**. Almost every confusing bug in your first few weeks will come down to this gap.

What is a text editor, and what will you use?

Code is just text, written in a plain text file — not a Word document, no formatting, no fonts, just characters. You write it in a **text editor** built for code (as opposed to Notepad or Word). Popular ones: Visual Studio Code ("VS Code," free, extremely common), PyCharm, Sublime Text. If you don't already have one, install VS Code before continuing — you'll be living in it for the next two weeks.

A Python file is a plain text file whose name ends in `.py` — for example `hello.py`. That file extension is how your computer (and you) recognize it as Python code rather than, say, a text document.

What is "the terminal," and why does everyone use it?

The **terminal** (also called the "command line," "console," or "shell") is a text-based way of telling your computer to do things, instead of clicking icons. Instead of double-clicking a program's icon, you type its name and press Enter. It looks intimidating the first time because it's just a blank window with a blinking cursor and no buttons — but it's just a text conversation with your computer: you type a command, press Enter, and the computer prints back a response.

On Windows, you already have one open in this environment (PowerShell). You'll type commands like:

```
python --version
```

and press Enter, and the terminal will print something back, like `Python 3.14.5`. That's the whole interaction model: type a command, press Enter, read the output, repeat.

Why not just use icons and menus? Because almost every professional coding tool — running your code, installing packages, using version control (Day 13), talking to servers — is built to be operated from the terminal first, with graphical interfaces built on top later (if at all). Getting comfortable typing commands is a foundational skill, not a workaround.

Running a Python script

Once you've saved a file called `hello.py` containing:

```
print("Hello, world!")
```

you run it from the terminal by navigating to the folder that file is in, then typing:

```
python hello.py
```

and pressing Enter. Python reads `hello.py` top to bottom, executes each instruction, and in this case prints `Hello, world!` to the terminal. That's the entire cycle you'll repeat constantly for the next two weeks: **write code in a file** → **run it from the terminal** → **read the output** → **fix something** → **run it again**.

`print(...)` is a built-in instruction ("function," a term you'll fully learn on Day 4) that displays whatever is inside its parentheses on the screen. You will use `print()` constantly, especially early on, to see what your code is actually doing at each step — it's the single most useful tool for understanding your own code while you're learning.

What is the Python REPL (the "interactive shell")?

Instead of writing a whole file, you can also start Python directly in the terminal by typing just:

```
python
```

and pressing Enter. This drops you into the **REPL** (Read-Evaluate-Print Loop) — an interactive mode where you type one line of Python at a time, press Enter, and immediately see the result, without saving anything to a file first:

```
>>> 2 + 2
4
>>> print("hi")
hi
```

The `>>>` is Python's prompt, telling you it's waiting for input — you don't type the `>>>` yourself. The REPL is perfect for quickly testing a small idea ("what does this expression actually return?") without the overhead of creating a file. You'll be told throughout this course to "try it in the REPL" — this is what that means. To leave the REPL, type `exit()` and press Enter, or press Ctrl+Z then Enter on Windows.

Reading an error message without panicking

You will see error messages constantly — this is normal and does not mean you broke something badly. It means Python tried to follow your instructions and hit a step it couldn't do. Here's an example and how to read it:

```
print("Hello"
```

Running this (notice the missing closing parenthesis) produces:

```
File "hello.py", line 1
  print("Hello"
    ^
SyntaxError: '(' was never closed
```

Read error messages from the **bottom up**:

- 1 The last line (**SyntaxError: ...**) tells you the *category* of problem and a short description — here, an opening `(` that was never closed.
- 2 The line above with `^` points at roughly *where* Python got confused.
- 3 **File "hello.py", line 1** tells you *which file and line* to go look at.

The category name before the colon (**SyntaxError**, **NameError**, **TypeError**, **ValueError**, and many more) is itself useful information — you'll learn what several of these specifically mean starting Day 6, but even now, "SyntaxError" tells you the problem is how you *typed* the code (missing punctuation, bad structure), separate from "the logic is wrong" errors that only show up once the code actually runs.

The skill to build immediately: read the error message fully before doing anything else. New programmers often see red text, panic, and start randomly changing code. Instead: read what it says, find the line number it points to, and think about what that line is actually telling the computer to do.

Indentation — Python's way of showing "this belongs inside that"

Many languages use curly braces `{ }` to show which instructions belong together (e.g., "everything inside this block belongs to this **if**"). Python instead uses **indentation** — consistent leading whitespace — to mean the same thing. This isn't just a style preference; in Python, indentation is part of the actual syntax and changing it changes what the code means.

```
if 5 > 3:
    print("five is bigger")    # indented -- this line is INSIDE the if
    print("still inside")     # also indented -- also inside the if
print("this always runs")    # NOT indented -- outside the if, always runs
```

The convention (and what your editor will do automatically) is 4 spaces per indentation level — never tabs mixed with spaces, since Python will treat them inconsistently and raise an **IndentationError**. You'll internalize this within the first couple of days since VS Code and most editors indent automatically after a line ending in `:`.

Comments — notes to humans that Python ignores

Anything after a `#` on a line is a **comment** — Python skips it entirely; it's there purely for a human reading the code later (including future-you).

```
# This calculates the area of a rectangle
area = width * height    # width and height must already be defined above
```

Good comments explain *why* something is done a certain way, not *what* the code does (well-written code should make the "what" obvious just by reading it) — you'll see this convention followed throughout every

lesson in this course.

A few words you'll see constantly, defined up front

- **Variable** — a name that refers to a value (full explanation Day 1).
- **Function** — a named, reusable block of instructions you can "call" (run) by name, optionally handing it some input (full explanation Day 4). `print(...)` is a function.
- **Argument** — a value you hand to a function when you call it. In `print("hi")`, `"hi"` is the argument.
- **String** — text data, written between quotes, like `"hello"` or `'hello'` (either quote style works; be consistent).
- **Method** — a function that belongs to a specific piece of data, called with a dot, like `"hello".upper()`.
- **Object** — a piece of data in Python (a number, a string, a list, anything). In Python, genuinely everything is an object — you'll see exactly what this means and why it matters on Day 1.
- **Bug** — a mistake in your code that makes it behave incorrectly (or not run at all). Not a moral failing — every programmer, at every level, produces bugs constantly; the skill is finding and fixing them methodically, not avoiding them entirely.
- **Debugging** — the process of finding and fixing a bug (full toolset on Day 12, but you'll practice informally from Day 1 onward).

What "run the file and check PASS/FAIL" means for these exercises

Starting Day 1, every `exercises.py` file has spots marked `# TODO: implement`, followed by `pass` (a Python keyword meaning "do nothing," used as a placeholder so the file doesn't error while you're still working on it). Below that, the file has code that calls your function with test inputs and prints `PASS` or `FAIL` depending on whether your answer was correct. Your job each day:

- 1 Open `exercises.py` in your editor.
- 2 Replace `pass` under each `TODO` with real code that makes the function do what its docstring (the text in `"""triple quotes"""` right under the `def` line) describes.
- 3 Save the file, then run it from the terminal: `python exercises.py`.
- 4 Read the `PASS/FAIL` output. Fix anything that says `FAIL`, save, and run again.
- 5 Once everything passes, compare your solution to `solutions.py` in the same folder.

This loop — write, run, read output, fix, run again — is, in essence, the entire day-to-day job of a programmer at any experience level. You're not just learning Python syntax this week; you're building that loop into a habit.

You're ready for Day 1

Go to `day01-basics/lesson.md` next. Whenever a new term shows up that wasn't defined here, the lesson will define it right there, the first time it's used — you're not expected to know anything beyond what's in this file yet.